X-CAVATE Original Refactoring Report

Detailed Line-by-Line Mapping:
`xcavate_11_30_25.py` (5,568 lines) → `xcavate/` Package (6,201 lines)

March 8, 2026

Contents

1 Overview **3**
1.1 Size Comparison . . . 3

2 Section-by-Section Mapping **3**
2.1 Imports and Setup . . . 3
2.2 Argument Parsing and Global Variables . . . 3
2.3 File Reading and Preprocessing . . . 4
2.3.1 Detailed Breakdown . . . 4
2.3.2 Key Algorithmic Changes . . . 4
2.4 Original Network Plot . . . 5
2.5 Interpolation . . . 5
2.5.1 Key Algorithmic Changes . . . 5
2.6 Graph Construction (Endpoints, Branchpoints, Adjacency) . . . 6
2.6.1 Detailed Sub-section Mapping . . . 6
2.6.2 Key Algorithmic Changes . . . 6
2.7 DFS Pathfinding . . . 6
2.7.1 Detailed Sub-section Mapping . . . 7
2.7.2 Key Algorithmic Changes . . . 7
2.7.3 Performance on Test Network (31,271 points) . . . 7
2.8 Pass Subdivision . . . 8
2.9 Gap Closure Pipeline . . . 8
2.9.1 Detailed Sub-section Mapping . . . 8
2.9.2 How the 54.6% Reduction Was Achieved . . . 8
2.10 Downsampling . . . 9
2.11 Multimaterial Processing . . . 9
2.11.1 Detailed Mapping . . . 9
2.12 Overlap (SM and MM) . . . 10
2.13 Custom Gap Closure Files . . . 10
2.14 Coordinate Output Files . . . 10
2.15 G-code Generation . . . 11
2.15.1 Detailed Block Mapping . . . 11
2.15.2 How the Reduction Was Achieved . . . 11
2.16 Print Instructions . . . 11
2.17 Plot Generation (All Variants) . . . 12

2.17.1 Original Plot Blocks 12

3 New Features (Not in Original) 12

4 Reduction Summary 12

5 Algorithmic Complexity Improvements 13

6 Design Pattern Changes 13

1 Overview

The original X-CAVATE was a single monolithic Python script with 3 user-defined functions and 30+ global variables. The refactored version is a modular package with 25 classes, ∼95 functions, dataclass configuration, abstract base classes, and spatial indexing. This document traces every section of the original code to its new location, documents what changed algorithmically, and quantifies the reductions.

1.1 Size Comparison

Metric	Original	Refactored	Change
Total lines	5,568	6,201	+633 (+11.4%)
Files	1	27	+26
Functions/methods	3	∼95	+92
Classes	0	25	+25
Global variables	30+	0	−30
G-code blocks (copy-pasted)	6	3 subclasses + 1 ABC	−3 dup. blocks
Plot blocks (copy-pasted)	5–7	2 parameterized funcs	−3–5 blocks
Coordinate output blocks	2 (SM + MM)	1 parameterized func	−1 block

Table 1: Overall size comparison.

2 Section-by-Section Mapping

2.1 Imports and Setup

Original: Lines 1–37 (37 lines)
Refactored: `xcavate/__init__.py` (3 lines), `__main__.py` (6 lines) — total 9 lines

Imports pandas, numpy, argparse, matplotlib, plotly; creates output directories.

What changed: Output directory creation moved to `XcavateConfig.ensure_output_dirs()` (4 lines in `config.py`). Imports are now per-module (each file imports only what it needs). Random seed removed (not used anywhere after initial set).

2.2 Argument Parsing and Global Variables

Original: Lines 38–174 (137 lines)
Refactored: `cli.py` (192 lines) + `config.py` (131 lines) — total 323 lines

What changed:

1. **30+ global variables replaced by `XcavateConfig` dataclass.** Every parameter is now a typed field with a default value and docstring, rather than a bare `variable = args.variable` assignment.

2. **Custom G-code file paths.** The original hardcoded paths like `startExtrusionCode = 'inputs/custom/start_extrusion_code.txt'` (lines 136–161). The refactored code uses `CustomCodes.load_from_dir(path)` which reads all 12 template files once from a configurable directory.

3. **New parameters added:**
   - `-algorithm` (dfs or sweep_line) for pluggable pathfinding
   - `-overlap_algorithm` (retrace or consecutive) for selectable overlap
   - `-output_dir` (configurable output location, was hardcoded to `outputs/`)
4. **Type safety.** Enums for `PrinterType`, `PathfindingAlgorithm`, `OverlapAlgorithm` replace raw integers and strings.

Example: original global variable declarations (lines 101–174):

```
multimaterial = args.multimaterial    # -> config.multimaterial
tolerance = args.tolerance            # -> config.tolerance
nozzle_OD = args.nozzle_diameter      # -> config.nozzle_diameter
nozzle_radius = nozzle_OD/2           # -> config.nozzle_radius (property)
scaleFactor = args.scale_factor       # -> config.scale_factor
printhead1_axis = 'A'                 # -> config.axis_1 (configurable)
startExtrusionCode = 'inputs/...'     # -> CustomCodes.load_from_dir()
```

2.3 File Reading and Preprocessing

Original: Lines 177–324 (148 lines)
Refactored: `io/reader.py` (298 lines)

Reads network file, removes blank lines/headers, writes preprocessed.txt/inlets.txt/outlets.txt, reads into pandas DataFrame, converts to numpy, scales, rounds, matches inlet/outlet nodes.

2.3.1 Detailed Breakdown

Old Lines	What It Does	Refactored Location	Lines
177–205	Strip blank lines, extract vessel headers, write `preprocessed.txt`	`read_network_file()` reader.py:29–89	61
206–234	Parse inlet/outlet file, write `inlets.txt`, `outlets.txt`	`read_inlet_outlet_file()` reader.py:92–157	66
238–268	Read into pandas, detect columns, convert to numpy	Merged into `read_network_file()`	incl.
279–305	Unit conversion (cm→mm), scaling, rounding	`preprocess_coordinates()` reader.py:172–226	55
307–324	Match inlet/outlet coords to nodes (exact `==`)	`match_inlet_outlet_nodes()` reader.py:229–298	70

Table 2: File reading detailed breakdown.

2.3.2 Key Algorithmic Changes

1. **No intermediate files.** The original wrote `outputs/graph/preprocessed.txt`, `inlets.txt`, and `outlets.txt` as intermediate steps. The refactored code processes everything in-memory, eliminating 3 file writes and 3 file reads.
2. **Inlet/outlet matching: exact float `==` vs KD-tree nearest-neighbor.**

- Original (lines 307–324): Triple-nested loop comparing `x == x`, `y == y`, `z == z` individually. Brittle with floating-point values.
- Refactored: `scipy.spatial.cKDTree` with `.query()` for $O(\log N)$ nearest-neighbor. Warns if distance > 5.0.

3. **Coordinate rounding: triple-nested Python loop vs vectorized numpy.**
   - Original (lines 292–305): `for i in range(len(points)):` $\rightarrow O(N \times C)$ Python-level iteration.
   - Refactored: `np.round(points, decimals=num_decimals)` — single vectorized call.
4. **Two-pass parsing.** The refactored reader scans the file twice: first to count vessels and points (for pre-allocation), then to fill the array. The original used Python list appending and pandas conversion.

2.4 Original Network Plot

Original: Lines 328–394 (67 lines)
Refactored: `viz/plotting.py:77-136` (`create_original_network_plot`, 60 lines)

Creates 3D Plotly scatter plot of pre-interpolation network, one trace per vessel, with slider. The function was extracted and parameterized. The original inline block was nearly identical to 4 other plotting blocks; the refactored version is a single reusable function.

Note: In the initial refactoring, `create_original_network_plot()` existed but was **never called** (dead code). This was fixed in the parity update by adding the call in `_write_plots()`.

2.5 Interpolation

Original: Lines 396–487 (92 lines)
Refactored: `core/preprocessing.py:45-181` (`interpolate_network`, 137 lines)

Densifies network so consecutive point spacing does not exceed nozzle radius.

2.5.1 Key Algorithmic Changes

1. **$O(N)$ single `np.concatenate` vs repeated `np.insert`.**
   - Original (lines 437–481): For each gap, calls `np.insert()` which copies the entire array. For a network gaining 30,000 points, this is $O(N^2)$ total copy operations.
   - Refactored: Builds a list of segment arrays, then calls `np.concatenate()` once at the end. $O(N)$ total.
2. **Flag column encoding.**
   - Original: Sets column 4 (or 5) to `500` for vessel start points, reuses the same column.
   - Refactored: Appends a dedicated flag column: `500` = vessel start, `400` = interpolated, `0` = original. Cleaner separation.
3. **Line count increase (+45 lines):** Due to more comprehensive docstrings, column-count handling, and the `get_vessel_boundaries()` helper function (68 lines).

2.6 Graph Construction (Endpoints, Branchpoints, Adjacency)

Original: Lines 489–906 (418 lines)
Refactored: `core/graph.py` (836 lines)

Finds endpoints, detects branchpoints via distance search, validates daughter pairs via parametric line test, builds adjacency graph.

2.6.1 Detailed Sub-section Mapping

Old Lines	Step	Refactored Function	Lines
489–540 (52)	Find endpoints per vessel	`_find_endpoints()` graph.py:162–203	42
541–559 (19)	Re-match inlets/outlets post-interpolation	Handled in pipeline	0
560–742 (183)	Find branchpoints (distance search + line test)	`_find_branchpoints()` + `_validate_daughter_pairs()` graph.py:210–540	331
828–906 (79)	Build adjacency graph + remove cross-edges	`_build_adjacency()` + `_remove_daughter_cross_edges()` graph.py:596–750	155
(inline)	Write `graph.txt`, `special_nodes.txt`	`write_graph()` + `write_special_nodes()` graph.py:757–835	79

Table 3: Graph construction sub-section mapping.

2.6.2 Key Algorithmic Changes

1. **Branchpoint detection: $O(N \times E)$ nested loops vs $O(N)$ KD-tree.**
   - Original (lines 575–625): For each endpoint, iterates through ALL points to find two nearest points outside the endpoint's vessel. $O(E \times N)$.
   - Refactored: Builds `cKDTree` once, queries `.query(point, k=10)` for k nearest neighbors, filters by vessel. $O(E \times k \times \log N)$.
2. **Parametric line test preserved.** The daughter validation logic is preserved but extracted into `_select_pair_by_line_test()`.
3. **Line count doubled (+418 lines):** Due to 12 separate functions (vs inline code), comprehensive docstrings, and additional helper functions for edge cases (e.g., `_brute_force_nearest()` fallback).

2.7 DFS Pathfinding

Original: Lines 911–995 (85 lines)
Refactored: `core/pathfinding.py` (620 lines)

Recursive DFS with collision checking to generate print passes — refactored into iterative DFS with KD-tree collision checking; also includes sweep-line alternative.

Old Lines	Function	Refactored Location	Lines
911–935 (25)	`is_valid()` — checks if printing a node blocks others below	`CollisionDetector.` `is_valid()` pathfinding.py:110–155	46
937–950 (14)	`find_lowest_unvisited()` — $O(N)$ linear scan	`_pop_lowest_unvisited()` with min-heap pathfinding.py:326–348	23
953–982 (30)	`dfs()` — recursive DFS	`iterative_dfs()` with stack pathfinding.py:351–407	57
983–995 (13)	Main loop calling DFS per pass	`DFSStrategy.` `generate_print_passes()` pathfinding.py:265–323	59
(new)	Sweep-line alternative	`SweepLineStrategy` pathfinding.py:414–560	147
(new)	Collision detector class	`CollisionDetector` pathfinding.py:48–200	153

Table 4: DFS pathfinding sub-section mapping.

2.7.1 Detailed Sub-section Mapping

2.7.2 Key Algorithmic Changes

1. **Recursive → Iterative DFS.** Original: Python recursive function that hits `RecursionError` on networks with >1,000 nodes after interpolation. Our test network (31,271 points) crashes at default limit. Refactored: Explicit stack-based DFS, no recursion limit.

2. **`is_valid()` collision check: $O(N)$ vs $O(k \log N)$.** Original: computes XY distance to ALL unvisited nodes. Refactored: `cKDTree.query_ball_point(xy, nozzle_radius)` finds only nearby nodes.

3. **`find_lowest_unvisited()`: $O(N)$ vs $O(\log N)$.** Original: iterates all nodes, finds minimum z among unvisited. Refactored: uses `heapq` min-heap sorted by z.

4. **DFS neighbor ordering.** Original: explores neighbors in `sorted(graph[node])` order with full recursion. Refactored: pushes neighbors onto stack in `reversed(sorted())` order. Can produce different pass compositions.

5. **New: Strategy pattern.** `PathfindingStrategy` ABC with `generate_print_passes()` method. DFS and sweep-line are interchangeable via `-algorithm` CLI flag.

2.7.3 Performance on Test Network (31,271 points)

Metric	Original	Refactored
Collision check	$O(N)$ per node	$O(k \log N)$ per node
Lowest unvisited	$O(N)$ per pass	$O(\log N)$ per pass
Recursion	Crashes at default limit	No recursion
Total DFS time	∼4–5 minutes	<1 second

Table 5: Pathfinding performance comparison.

2.8 Pass Subdivision

Original: Lines 997–1105 (109 lines)
Refactored: `core/postprocessing.py:11-84` (`subdivide_passes`, 74 lines)
Reduction: −35 lines (−32.1%)

Splits passes at DFS backtrack points; ensures bottom-up ordering.

What changed: Logic is identical. Reduction comes from removing changelog file writes and more concise Python idioms (e.g., `print_passes[i].index(bp)` instead of manual iteration).

2.9 Gap Closure Pipeline

Original: Lines 1107–3200 (2,094 lines)
Refactored: `core/gap_closure.py` (951 lines)
Reduction: −1,143 lines (**−54.6%**)

This is the single largest reduction in the refactoring.

6-part iterative gap closure: disconnect detection, condition 0 (vessel gaps), conditions 1–3 (branch-point connections), final closure.

2.9.1 Detailed Sub-section Mapping

Old Lines	Section	Refactored Function	Old	New
1107–1340	Changelog 0 (disconnect detect)	`find_disconnects()` (shared)	234	244
1341–1532	Condition 0 (vessel gaps)	`close_gaps_condition0()`	192	104
1534–1758	Changelog 1 (re-detect)	`find_disconnects()` (reused)	225	0
1776–1906	Branchpoint cond. 1 (backwards)	`close_gaps_branchpoint` (dir=`"backwards"`)	131	117
1908–2137	Changelog 2+3 (re-detect ×2)	`find_disconnects()` (reused ×2)	230	0
2138–2259	Branchpoint cond. 2 (forwards)	`close_gaps_branchpoint` (dir=`"forwards"`)	122	0
2260–2488	Changelog 4 (re-detect)	`find_disconnects()` (reused)	229	0
2489–2572	Branchpoint cond. 3 (parent)	`close_gaps_branchpoint` (dir=`"parent_to_nbr"`)	84	55
2573–2802	Changelog 5 (re-detect)	`find_disconnects()` (reused)	230	0
2803–3099	Final gap closure	`close_gaps_final()` + `remove_artifact_passes()`	297	107
3100–3200	Final cleanup	`run_full_gap_closure_pipeline()`	101	151

Table 6: Gap closure pipeline detailed mapping.

2.9.2 How the 54.6% Reduction Was Achieved

1. **Disconnect detection: 6 copies → 1 function.** The original copied the ∼230-line disconnect detection block **6 times** (changelogs 0–5), each nearly identical. The refactored

code has a single `find_disconnects()` function (244 lines) called 6 times. This alone saves ∼**1,150 lines**.

2. **Branchpoint conditions: 3 similar blocks → 1 parameterized function.** Conditions 1, 2, and 3 had nearly identical structure but differed in search direction. The refactored `close_gaps_branchpoint()` takes a `direction` parameter (`"backwards"`, `"forwards"`, `"parent_to_neighbor"`).

3. **Pipeline orchestrator.** `run_full_gap_closure_pipeline()` (151 lines) replaces the implicit sequencing of inline blocks with an explicit 12-step pipeline with logging at each stage.

2.10 Downsampling

Original: Lines 3216–3310 (95 lines)
Refactored: `core/postprocessing.py:87-129` (`downsample_passes`, 43 lines)
Reduction: −52 lines (−54.7%)

Keeps every Nth node while preserving first/last, branchpoints, endpoints.

Reduction source: The original included an inline plot generation block (∼50 lines) that was moved to `_write_plots()` in the pipeline.

2.11 Multimaterial Processing

Original: Lines 3320–3695 (376 lines)
Refactored: `core/multimaterial.py` (98 lines) + `pipeline.py:352-434` (83 lines) — total 181 lines
Reduction: −195 lines (−51.9%)

Classifies passes by artven type, swaps noise, subdivides at material boundaries, computes speeds, generates MM plot.

2.11.1 Detailed Mapping

Old Lines	What It Does	Refactored Location	Old	New
3320–3340	Extract artven values per pass	Part of `_subdivide_by_material()`	21	incl.
3357–3432	Noise swapping (outlier artven at boundaries)	`_subdivide_by_material()` pipeline.py:354–371	76	18
3433–3583	Find break points, subdivide, reassemble	`_subdivide_by_material()` pipeline.py:372–434	151	65
3610–3620	Classify passes (last node's artven)	`classify_passes_by_material()` multimaterial.py:11–41	11	31
3629–3695	MM plot generation	`_write_plots()` + `create_network_plot()`	67	1
3697–3727	Speed: `flow / (`πr^2`)`	`compute_radius_speeds()` multimaterial.py:44–75	31	32

Table 7: Multimaterial processing detailed mapping.

Reduction sources: MM plot block (67 lines) replaced by 1-line call to parameterized function. Artven extraction simplified (no separate dictionary). Subdivision logic more concise with Python idioms.

2.12 Overlap (SM and MM)

Original: SM: Lines 3729–3806 (78 lines); MM: Lines 3880–3959 (80 lines)
Refactored: `core/postprocessing.py:132-242` (111 lines total)
Reduction: −47 lines

Finds passes ending on shared nodes in earlier passes, retraces backwards to add overlap nodes.

What changed:

1. **SM and MM overlap: 2 near-identical blocks → 1 function.** The original had separate SM and MM blocks with the same logic. The refactored `add_overlap()` works on any pass dictionary.
2. **Two algorithms available.** The `retrace` algorithm matches the original. The `consecutive` algorithm is a new faster alternative that only checks adjacent pass pairs.

2.13 Custom Gap Closure Files

Original: Lines 3808–3876 (69 lines)
Refactored: `pipeline.py:437-462` (`_load_gap_extensions`, 26 lines)
Reduction: −43 lines (−62.3%)

Reads pass indices and delta vectors from extension files.

Reduction source: The original had separate SM and MM reading blocks. The refactored code has a single function called with a suffix parameter (`"SM"` or `"MM"`).

2.14 Coordinate Output Files

Original: SM: Lines 4090–4251 (162 lines); MM: Lines 4253–4405 (153 lines)
Refactored: `io/writer.py` (148 lines)
Reduction: −167 lines (−53.0%)

Writes x, y, z, radius, speed, and combined coordinate files per pass.

How the reduction was achieved:

1. **SM and MM: 2 blocks → 1 parameterized function.** `write_pass_coordinates()` handles both SM and MM by parameter.
2. **Per-column output: repeated inline loops → `_write_column_file()` helper.** The original had separate loop blocks for x, y, z, radius, and artven output.
3. **Speed output separated.** `_write_speed_file()` handles the different speed dictionary format.

2.15 G-code Generation

Original: Lines 4409–5486 (1,078 lines across 6 blocks)
Refactored: `io/gcode/` package (713 lines across 4 files)
Reduction: −365 lines (**−33.9%**)

This is the second largest reduction in the refactoring.

2.15.1 Detailed Block Mapping

Original Block	Old	Refactored Class	New	Δ
SM Pressure (4409–4506)	98	`PressureGcodeWriter`	∼70	−28
SM Positive Ink (4508–4629)	122	`PositiveInkGcodeWriter`	∼85	−37
SM Aerotech (4632–4711)	80	`AerotechGcodeWriter`	∼60	−20
MM Pressure (4713–4974)	262	`PressureGcodeWriter`	∼70	−192
MM Positive Ink (4976–5277)	302	`PositiveInkGcodeWriter`	∼95	−207
MM Aerotech (5279–5486)	208	`AerotechGcodeWriter`	∼90	−118
(not in original)	0	`GcodeWriter` ABC	183	+183
Total	**1,072**		**713**	**−365**

Table 8: G-code generation block mapping. SM/MM pairs share one class each.

2.15.2 How the Reduction Was Achieved

1. **Abstract base class `GcodeWriter` (183 lines).** Extracts the shared iteration loop (header → for each pass → for each node → footer), gap extension logic, and file I/O. All 6 original blocks had this same loop structure copy-pasted.
2. **SM/MM merged into single classes.** Each original pair (SM + MM) was merged into one class with `if config.multimaterial:` branches.
3. **Custom G-code loading: per-pass file open → load-once.** Original: opened and read the same file on **every pass start** (potentially 40+ times). Refactored: `CustomCodes.load_from_dir()` reads all 12 template files once into memory at startup.
4. **Shared methods eliminate duplication:**
   - `_write_header()` — 1 implementation instead of 6 inline blocks
   - `_write_footer()` — 1 implementation instead of 6 inline blocks
   - `_write_network()` — 1 shared loop instead of 6 copy-pasted loops
   - `_write_custom()` — 1 helper instead of repeated `with open(...) as f:`

2.16 Print Instructions

Original: Lines 5488–5568 (81 lines)
Refactored: `pipeline.py:516-619` (`_print_instructions`, 104 lines)

Computes bounding box, centering positions, prints SM/MM operator instructions.

What changed: Logic is identical. Line increase due to function signature, docstring, and string formatting across multiple lines for readability. The instructions themselves are character-for-character identical (validated by test output comparison).

Note: This function was **completely missing** from the initial refactoring and was added during the parity fixes.

2.17 Plot Generation (All Variants)

Original: 5–7 inline blocks totaling ∼335 lines
Refactored: `viz/plotting.py` (137 lines) + calls in `pipeline._write_plots()` (49 lines) — total 186 lines
Reduction: −149 lines (−44.5%)

2.17.1 Original Plot Blocks

Lines	Plot	Refactored
328–394 (67)	Original network	`create_original_network_plot()` (60 lines)
3100–3155 (56)	Final SM passes	`create_network_plot()` (62 lines, shared)
3258–3310 (53)	Downsampled SM	`create_network_plot()` (reused)
3629–3695 (67)	MM passes (art./venous)	`create_network_plot(colors=...)` (reused)
3965–4018 (54)	SM overlap	`create_network_plot()` (reused)

Table 9: Original plot blocks and their refactored equivalents.

Reduction mechanism: 5 copy-pasted plot blocks replaced by 2 parameterized functions. Each original block was ∼55–67 lines of nearly identical Plotly code with minor variations (title, color, data source).

3 New Features (Not in Original)

Feature	Location	Lines
Streamlit GUI	`gui/app.py`	792
Sweep-line pathfinding	`core/pathfinding.py`	147
Nearest-neighbor pass reordering	`core/postprocessing.py`	46
Spatial index module	`spatial/index.py`	120
Test suite	`tests/` (5 files)	501
`XcavateConfig` dataclass	`config.py`	131
Progress callbacks	`pipeline.py`	∼10
Structured logging	Throughout	∼20

Table 10: New features added during refactoring.

Total new code not derived from original: ∼1,767 lines. This accounts for the overall increase from 5,568 to 6,201 lines despite significant reductions in existing code.

4 Reduction Summary

Key insight: The ported code is 16.5% shorter despite being more modular, better documented, and having more robust algorithms. The overall 11.4% increase is entirely due to new features (GUI, sweep-line, tests, spatial indexing, config system).

Section	Original	Refact.	Change	%
Gap closure pipeline	2,094	951	−1,143	−54.6%
G-code generation (6 blks)	1,078	713	−365	−33.9%
Plot generation (5–7 blks)	∼335	186	−149	−44.5%
Coordinate output (SM+MM)	315	148	−167	−53.0%
Overlap (SM+MM)	158	111	−47	−29.7%
Downsampling + plot	95	43	−52	−54.7%
Multimaterial processing	376	181	−195	−51.9%
Gap extension file loading	69	26	−43	−62.3%
File reading/preprocessing	148	298	+150	+101.4%
Graph construction	418	836	+418	+100.0%
DFS pathfinding	85	620	+535	+629.4%
Subdivision	109	74	−35	−32.1%
Print instructions	81	104	+23	+28.4%
Args/config/setup	174	332	+158	+90.8%
Sub-total (ported code)	**5,535**	**4,623**	**−912**	**−16.5%**
New features (GUI, tests, ...)	0	1,578	+1,578	N/A
Grand total	**5,568**	**6,201**	**+633**	**+11.4%**

Table 11: Reduction summary across all sections.

5 Algorithmic Complexity Improvements

Operation	Original	Refactored
Collision detection (`is_valid`)	$O(N)$ per call	$O(k \log N)$ per call
Lowest unvisited node	$O(N)$ per pass	$O(\log N)$ amortized
Interpolation (N insertions)	$O(N^2)$ total copies	$O(N)$ single concatenation
Branchpoint detection	$O(E \times N)$ nested loops	$O(E \times k \times \log N)$ KD-tree
Inlet/outlet matching	$O(N \times I)$ exact cmp	$O(I \times \log N)$ KD-tree
Coordinate rounding	$O(N \times C)$ Python loop	$O(1)$ vectorized numpy
Custom G-code reading	$O(P \times F)$ file opens	$O(F)$ load once

Table 12: Algorithmic complexity improvements. $N \approx 31{,}000$ (points), $E \approx 22$ (endpoints), $I = 2$ (inlets/outlets), $P \approx 45$ (passes), $F = 12$ (template files), $C = 3$–5 (columns), $k \approx 5$–10 (nearby neighbors).

6 Design Pattern Changes

Pattern	**Original**	**Refactored**
Configuration	30+ global variables	`XcavateConfig` dataclass with typed fields
G-code generation	6 copy-pasted blocks with inline conditionals	ABC + 3 subclasses (Template Method pattern)
Pathfinding	Single recursive function	Strategy pattern with pluggable algorithms
Plot generation	5–7 inline Plotly blocks	2 parameterized functions (DRY)
File I/O	Inline `open/read/write` throughout	Dedicated reader/writer modules
Coordinate output	2 copy-pasted blocks (SM + MM)	1 parameterized function
Disconnect detection	6 copy-pasted blocks	1 function called 6 times
Branchpoint conds	3 similar blocks	1 parameterized function with direction parameter
Spatial queries	Brute-force distance computation	KD-tree spatial indexing
Error handling	None (crashes on bad input)	Validation, warnings, graceful errors
Progress reporting	`print()` statements	`logging` module + GUI progress callbacks
Entry points	Script execution only	CLI + GUI + library import

Table 13: Design pattern changes from original to refactored code.